

A Modular Approach for Engineering Situated Multi-agent Systems with Self-adaptation

Delabie, Mathijs
KULeuven Kulak
Kortrijk, Belgium

mathijs.delabie@student.kuleuven.be

Weyns, Danny
KULeuven Kulak
Kortrijk, Belgium

danny.weyns@kuleuven.be

Abstract—Self-adaptation (SA) is a promising approach to manage the complexity of modern software systems. Multi-agent systems (MAS) are a type of system where SA can be helpful. Therefore, SA-MAS has been proposed. In this paper, we concentrate on situated MAS. In most approaches for MAS, there is little attention for modularity. When designing a SA-MAS, it is useful to be able to add, remove and change components without significant impact on other components. In this paper, we present a modular approach for situated MAS with self-adaptation. The approach consists of expanding the model of self-adaptation with a MAPE loop, with a metabehaviour. The metabehaviour separates the overall agents behaviour in smaller behaviours. An agent self-adapts its behaviour with the use of a MAPE-loop. To evaluate this approach, we use it to design a MAS set in the packet-world and add behaviour to test the modularity. We also compare its modularity with other approaches.

Keywords—Multi-agent systems, MAS, situated MAS, situated agents, self-adaptation, modularity, modular approach.

I. INTRODUCTION

A multi-agent system (MAS) is a decentralized system composed of multiple interacting independent agents that live in an environment. Multi-agent systems can be used to solve problems that are difficult or impossible for an individual agent or a monolithic system to solve.

Engineering modern software systems such as multi-agent systems (MAS) can be a difficult and complex task. One of the reasons lies in the complexity to combine multiple concerns that a MAS has to address, such as system functionality, coordination, robustness, etc. A well-recognized approach to manage system complexity is the use of self-adaptation [9], [4]. Self-adaptation extends a system with support to monitor and adapt itself to realize a concern of interest (optimization, fault-tolerance, etc.). The key idea behind self-adaptation is complexity management through separation of concerns.[3] There are formal models for self-adaptation in MAS [5], [9] and self-adaptation has been applied to MAS in a number of research and engineering efforts, some examples are [12], [7], [8]. A common approach to realize self-adaptation is by a MAPE feedback loop [4], [13]. This is an external loop added to a managed system. It monitors the managed system and the environment, analyses the incoming data, creates a plan to change the managed system and executes this plan.

For software designers, modularity is an important aspect. A modular design means that the different components are loosely coupled and that adding, removing and changing components has little or no effect on other components. To address

modularity in MAS, modular interpreted systems (MIS) have been proposed [17]. These systems are a class of models that should achieve separation of the interference between agents from the local processing within agents.[17] In SA-MAS, there is no substantial research for modularity. However there are methods for role modelling that promise reuse of design [16]. In this paper we don't work with organizations in which roles are used. For this reason we will not distinguish roles and behaviours.

In this paper, we present a modular approach for situated MAS with self-adaptation. The approach consists of expanding the model of self-adaptation with a MAPE loop, with a metabehaviour. It uses the idea of separation of concerns to separate complex behaviour in a metabehaviour and smaller behaviours. The approach consists of a managed system with behaviours, a metabehaviour and a MAPE feedback loop. A metabehaviour is a behaviour about the behaviour. It determines which behaviour is used by an agent. The MAPE loop receives data from the managed system and the environment and plans a change of behaviour in the metabehaviour. This approach contributes to modularity in situated SA-MAS [3]. It should be possible to add, remove and change behaviours in the managed system without having to change a lot in the metabehaviour and the MAPE loop.

To evaluate this approach, we use it to design a MAS set in the packet-world [11]. We then extend the MAS with extra behaviour for an extended packet-world. We do this to test the modularity. We also compare the approach with other approaches on modularity. They are graded on how much extra changes need to be done.

The remainder of this paper is structured as follows. In Section II, we go a little deeper into situated MAS. Section III introduces the background information which exists of the MAPE loop [4] and timed automata [1]. Section IV presents the modular approach and in Section V we apply the approach with a case example. In Section VI we reflect on the case example and compare the approach with other approaches and finally, we draw conclusions in Section VII.

II. SITUATED MAS

MAS can be divided in two kinds of systems, cognitive MAS and situated MAS. The difference between the two are the agents. Cognitive agents can have a lot of individual intelligence. Which introduces more complexity than situated agents [6]. Because of the nature of both kinds of MAS, they have different advantages in solving problems. Cognitive

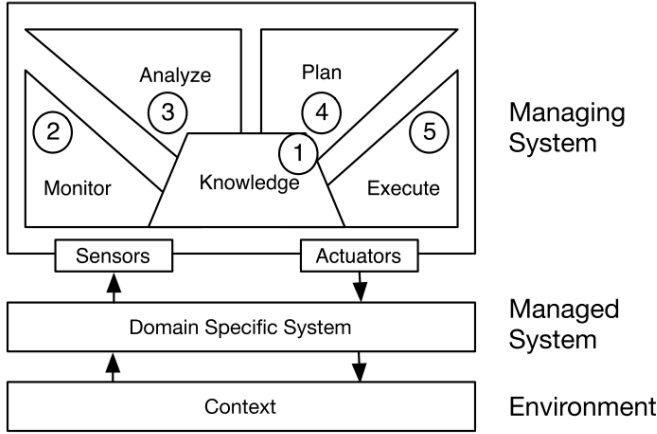


Figure 1. MAPE feedback loop [4], [2]

MAS are useful for problems that need complex calculations or long-term planning, to solve it. Situated MAS have an advantage in problems with simple goals, where interactions are more important than the cognitive capabilities of the agents. In this paper we concentrate on situated MAS. Situated agents are entities that encapsulate their own behaviour and maintain their own state. [11] They have local view of the environment, they can only see and interact with objects and other agents in this view. Situated agents act in the present, "here" and "now". Intelligence in a situated MAS originates from the interactions between the agents, rather than from their individual capabilities.[11]

III. BACKGROUND

In this section we present background information that is needed to understand the modular approach.

A. MAPE feedback loop

The MAPE structural design for self-adaptive systems was introduced by IBM in 2003 [14]. It contains five components as you can see on figure 1. These five components make up the design for a self-adaptive system. The first component is the Knowledge component, it is marked with (1) in figure 1. Its function is to collect and keep track of the knowledge in the MAPE feedback loop. This can be knowledge of the environment or the managed system, but also knowledge of the inner workings of the MAPE loop. All other four components have access to this knowledge. The second component is the Monitor (2). The monitor receives information from outside the loop via probes or sensors. It does some pre-processing if necessary and updates the knowledge. When the monitor is done it triggers the next component, Analyse (3). This component analyses the newly update knowledge and determines if the current state of the managed system is satisfied. After analysing the Plan (4) component is triggered. This component creates a plan on basis of the analysis of the previous component. This plan consists of how the managed system should change to be satisfied. The last component is Execute (5). It is responsible for the execution of the plan. In figure 1, you can see two additional components, sensors and actuators. The sensors or probes probe the external information

Figure	Name	Description
	State	States of behaviors are represented by circles and (optionally) annotated in red on top of the associated state.
	Committed State	Committed states are represented with circles containing the <i>c</i> character. Committed states must be left without time consumption.
	Urgent State	Urgent states of behaviors are represented with circles containing the <i>u</i> character. Urgent states must be left as soon as exiting conditions are found (normally defined by conditions on outgoing transitions).
	Initial state	Initial States of behaviors are represented by double-lined circles. There must be one unique <i>Initial State</i> per automaton, specifying the behavior state when the system starts.
	Invariants	Invariants that need to be satisfied in certain states are annotated in purple under the related state.
	Transition	Transitions between two states are represented by directional arrows, showing the origin and destination of the transition.
	Conditions	Conditions to enable transitions between states are annotated in green under the related transition.
	Signal	Signals used for communication between behaviors are annotated dark blue over the associated transition.
	Function	Functions associated with behaviors are annotated in light blue under the associated transition.

Figure 2. Conventions in Timed Automata figures [2]

to the monitor component. The actuators or effectors are used by the execute component to execute the created plan.

B. Timed automata

In this paper we use timed automata [1] as formal language to model the environment, managed system with the metabehaviours and behaviours and the different components of the MAPE loop. A timed automaton is a finite-state machine extended with clock variables that models a behaviour. Clock variables are used to synchronize behaviours. With timed automata, you can easily design rigorous models. Automata can communicate through channels, where the sender behaviour can send signals *S* (represented by the exclamation sign: channel!) in order to synchronize with the receiver behaviour listening for signal *S* (represented by the question mark sign: channel?). Timed Computation Tree logic (TCTL) [1] allows designers to specify properties of timed automata. The following property (1) specifies that when the analyser is unsatisfied, the planner should plan to add resources.

$$Analyse(X).Unsatisfied \rightarrow Plan(X).AddResource \quad (1)$$

In this paper we use UPPAAL [15] to model timed automata. In UPPAAL behaviour specifications can be complemented with expressions specified in a C-like language to define data structures and functions. We follow UPPAAL symbol conventions for the description of the behaviour templates, presented in figure 2. Besides designing the model, UPPAAL can also be used to simulate and to verify properties of your model. The model checking engine of UPPAAL is designed to check a subset of TCTL formula [18] for networks of timed automata. Local properties can be checked locally on a state, i.e. boolean expressions over predicates on locations and integer variables, and clock constraints. [1] In UPPAAL a model can be directly checked on deadlocks with the following example (2). This example checks if there is invariantly $A[]$ no deadlock.

$$A[] \text{ not deadlock} \quad (2)$$

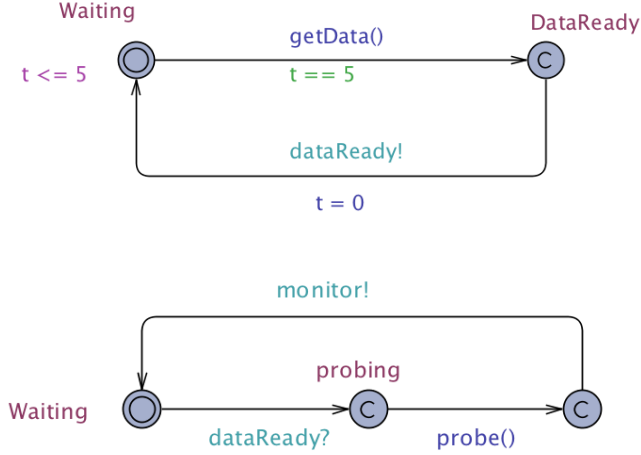


Figure 3. Examples of timed automata with sendbehaviour and receivebehaviour

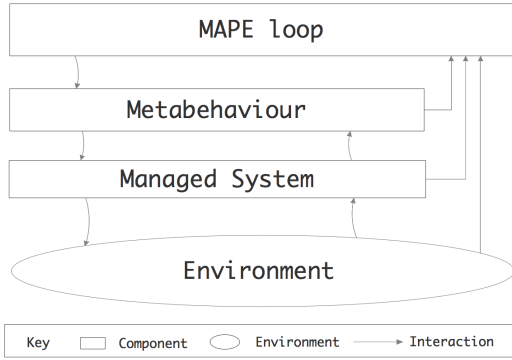


Figure 4. High level model of an agent for the modular approach

This second example (3) checks if the previously mentioned property is always true.

$$A \models \text{forall}(X) \text{Analyse}(X). \text{Unsatisfied imply} \\ \text{Plan}(X). \text{AddResource} \quad (3)$$

In figure 3, there are two examples of timed automata. The first (upper) is a detector that waits to detect data and sends a signal when data is ready. This is sendbehaviour, the detector sends a signal via the *dataReady* channel. The second (lower) example, a probe, receives this signal and probes the data detected by the detector. Besides the receivebehaviour via the *dataReady* channel, the probe also has sendbehaviour via the *monitor* channel.

IV. MODULAR APPROACH

In this section, we present a modular approach for situated MAS with self-adaptation that allows relatively easy changes. We start with a high level model of the approach.

A. High level model

In a multi-agent system, an agent has a managed system that interacts with the environment it lives in. As seen in

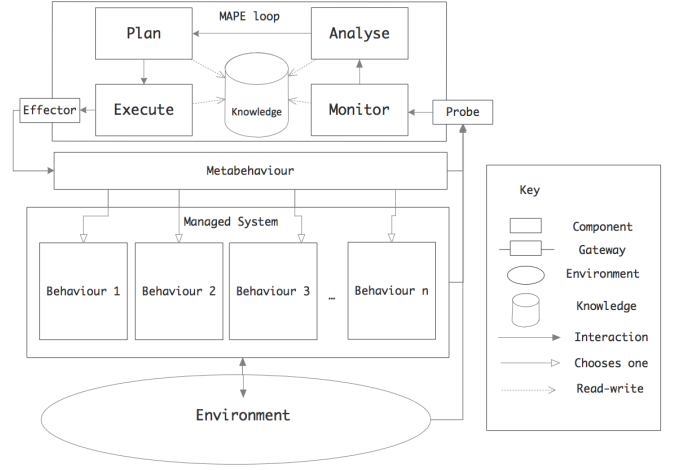


Figure 5. General model of an agent for the modular approach

figure 4 this managed system can perceive and change the environment on basis of a certain behaviour. The modular approach adds an extra component above the managed system, the metabehaviour. The metabehaviour decides which behaviour the agent uses. To change the behaviour of an agent, the metabehaviour needs to decide it changes behaviour, this happens via a MAPE loop. This loop receives information from the environment, the managed system and the metabehaviour. It determines if and to which behaviour the metabehaviour should change.

B. General model

In the general model two components are displayed in more detail (figure 5). The managed system contains multiple behaviours. One of these behaviours is chosen by the metabehaviour. This is the behaviour of the agent. All other behaviours do nothing until the metabehaviour has chosen one of them. The MAPE loop is composed out of the MAPE elements and two gateways. The gateways are the only external connections of the MAPE loop. The knowledge is the main database for the MAPE loop. All components can read and write to this database, but the knowledge has no connection to the outside of the MAPE loop. The knowledge collects data from the managed system, the environment, the working data and the concerns which determine the goals of the MAPE loop. The probe gateway probes data into the MAPE loop to the monitor. It receives the probed data, potentially does some pre-processing and updates the knowledge. After the analyser is triggered by the monitor, it analyses the data in the knowledge. The analysis can be one of two things: the metabehaviour is satisfied or the metabehaviour should change its behaviour. After analysing the planner is triggered and plans the potentially change in behaviour. If there is a plan created to change the behaviour, the execute is triggered. It uses the effector gateway to change the behaviour the metabehaviour uses.

C. Templates

In this subsection we go a little deeper into the components of the MAPE loop. These components have templates based on the MAPE-K formal templates [2]. The templates in this

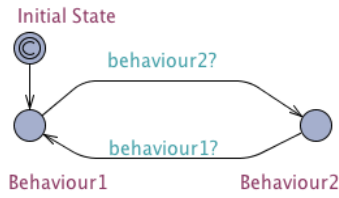


Figure 6. Template model for metabebehaviour with 2 behaviours



Figure 7. Template model for analyse-component for metabebehaviour with 2 behaviours [2]

paper are for a metabebehaviour with two behaviours. In figure 6, there is a template model for a metabebehaviour with two behaviours. Each behaviour is represented by a node and they are connected with edges who determine how a metabebehaviour can change from one behaviour to another. There is an extra node for the initial state to make sure there is always an initial state. The approach uses the monitor template in [2]. It gets the data from the probe, pre-processes it, updates the knowledge and triggers the analyser. The analyser (figure 7) than analyses the data and reports that it is satisfied or that the metabebehaviour should change behaviour. If the analyser was satisfied the planner does nothing. However, the planner creates a plan to change behaviour is necessary. The last component is the execute component (figure 8). It is triggered when the planner has a plan ready. When it is executed the metabebehaviour changes its behaviour according to the channel that has been called in the executor.

Next to behaviour specification templates, you can also find property specification templates in [2]. These templates are divided in three different groups or verification scopes (figure 9). In the first group, templates focus on providing

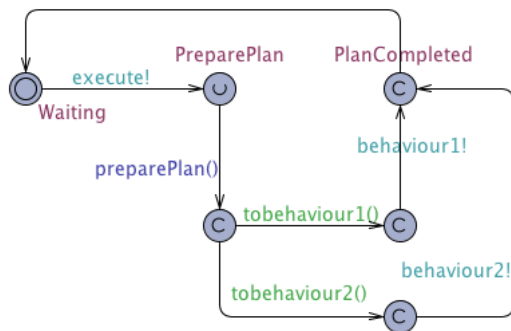


Figure 8. Template model for execute-component for metabebehaviour with 2 behaviours [2]

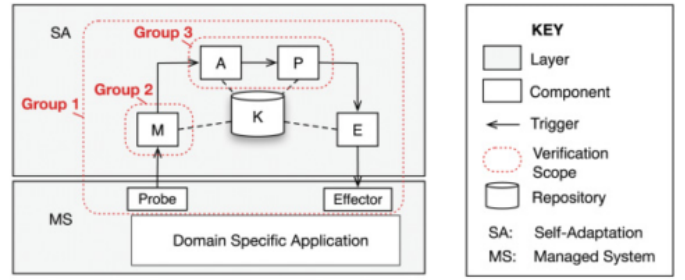


Figure 9. Three different property groups [2]

evidence about the realization of the self-adaptation goals.[2] In the second group, templates focus on verification of properties of individual MAPE components, independently of the other components.[2] In the third group, templates focus on interactions between MAPE components.

D. Framework

An extra contribution of this paper, is a framework for the modular approach for situated SA-MAS. This framework can be used to implement the MAPE loop and its components. In the following two listings you can see a part of code of the framework and an implementation. The first listing is the analyse-component in the framework. It specifies an abstract analyse method and links to the knowledge and to the planner. The second listing is an implementation of the analyse method. It first analyses the data in the knowledge, then sets the *isSatisfied* variable in the knowledge and lastly triggers the planner.

Listing 1. Framework of analyse-component

```
// analyses the knowledge data
public abstract void analyse();

// MAPE LINKS
private Knowledge knowledge;
private Planner planner;
```

Listing 2. Implementation of analyse method

```
public void analyse(){
    boolean isSatisfied
        = (seeChange && !packetinsight)
        || (!seeChange && packetinsight);
    getKnowledge().setVariable("isSatisfied",
        isSatisfied);
    getPlanner().plan();
}
```

E. Modularity

To make this model modular, we look at three different kinds of changes that can profit of modularity. These three changes are adding, removing and changing a behaviour. In a perfect modular model, these changes would have no effect on other components.

For adding a behaviour, we need to simply add a behaviour to the set of behaviours that the metabebehaviour can use. The

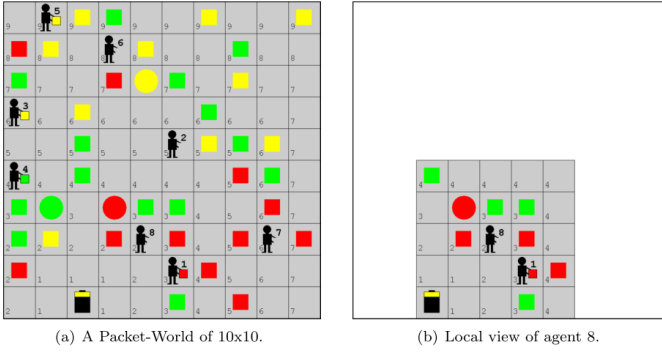


Figure 10. Example of the packet-world [11]

metabehaviour has to make a minor change by adding a node and edges for the new behaviour. The major changes happen in the MAPE loop. The amount of changes are dependent on the amount of edges that have been added, because the analysis of to which behaviour to change depends on these edges. The second change is removing a behaviour. The behaviour is removed from the set of behaviours and the node in the model for the metabehaviour is removed. The edges can be removed or the removed node can be replaced by an other already existing node. For the first case, all paths leading to removed edges have to be removed. In the second case, these paths may remain in the MAPE-loop. Only some minor changes are needed, concerning the new source or goal node of the edge. The last change is changing a behaviour. Again, there are two possibilities. The first possibility is that the model for the metabehaviour does not have to be changed. The only change needed is a behaviour being replaced by another behaviour. The other possibility is that the model for the metabehaviour does have to be changed. This means that certain edges can be added and removed. The amount of changes needed in the MAPE loop again depends on the amount of edges added, changed and removed. This last case can in worst case be seen as first removing a behaviour and then adding another behaviour.

If an existing implementation made with this approach, needs to be modified, a designer needs to follow these steps:

- 1) *Change model*: The designer changes the model for his MAS in one of the previous ways.
- 2) *Verify model*: Before using the model, it needs to be verified based upon the wanted properties.
- 3) *Change implementation*: After the model is finished, the implementation can be changed according to the model.
- 4) *Test implementation*: As last step, the implementation is tested on the supposed properties.

This approach is different from activForms [5], where the goals can be changed in runtime.

V. CASE EXAMPLE

This section is the first part of the evaluation, we look at a case example. This case uses the modular approach to design a metabehaviour. The models were made with UPPAAL [15] and with the MAPE-K formal templates [2].

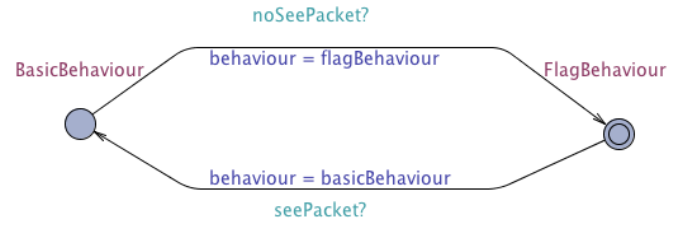


Figure 11. Model for single-room metabehaviour with basic- and flagbehaviour

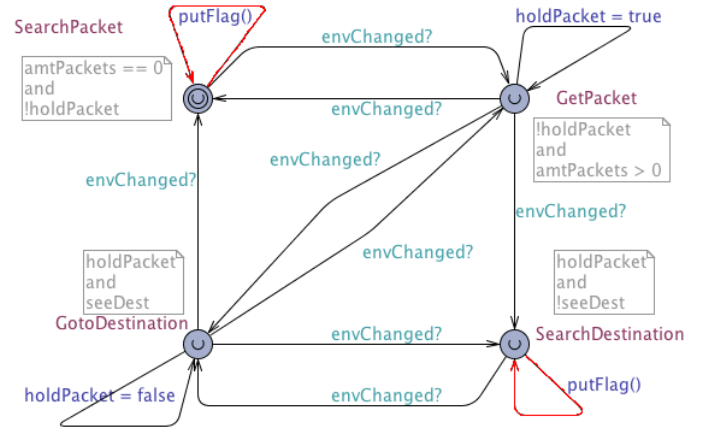


Figure 12. Model for basicbehaviour (without red edges) and flagbehaviour (with red edges)

A. Packet-world

The packet-world is an example of a situated MAS, we use the packet-world as an example of the modular approach in this case. The packet-world consists of a rectangular grid. Located in this grid are coloured packets and destinations. The agents that live in the packet-world and compose the multi-agent system in this world, need to bring the packets to a destination with the same colour. Figure 10(a) shows an example of a packet-world with a size of 10x10 with 8 agents. Coloured rectangles symbolize packets that can be manipulated by the agents and circles symbolize destinations. An agent in the packet-world is limited by its view. It can only see a part of the world. In figure 10(b), the view of an agent is presented. The view-range of this agent is 2. Agents can interact with the world in numerous ways. The standard interactions consist of picking up a packet and putting it at the right destination. The agents can be extended with other behaviours, such as flags and pheromones [11]. Also the world itself can be extended with extra items such as larger packets and energy stations. Agents in the packet-world have the ability to communicate. They can send messages or even pass a packet to another agent.

B. Single room Packet-World

A single room packet-world is the standard packet-world. In this case example we design a model for a metabehaviour in this basic packet-world. The metabehaviour has two behaviours, basic and flag behaviour. The basic behaviour is used when the agent sees packets and the flag behaviour is used when the agent doesn't see any packets.

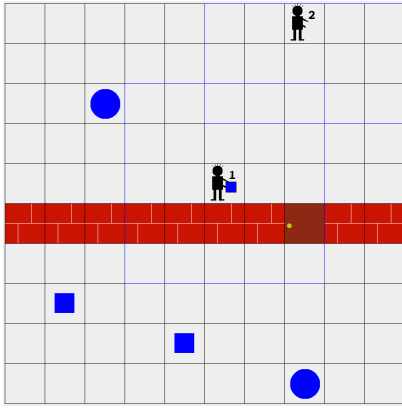


Figure 14. An example of a packet-world with rooms

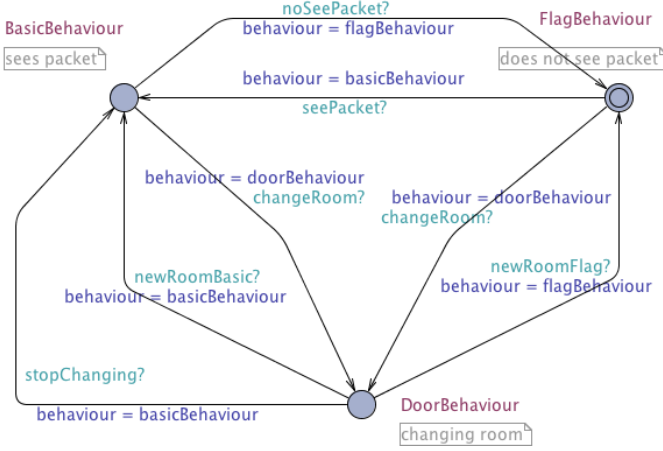


Figure 15. Model of metabehaviour for multiple room packet-world

behaviour, the door behaviour. In figure 14 you can see an example of a multiple room packet-world with two rooms. An agent can go from one room to another by going through a door. Agents can walk through doors, but they can not see through them. The difficulty introduced by these rooms is that an agent can get stuck searching in one room while the remaining packets are in another room. Therefore our previous metabehaviour needs extension.

A model for the new door behaviour can be found on figure 16. An agent with door behaviour will first search for a door. When it sees a door it will change its action to going through the door. When the agent eventually changed room, it starts searching for a door all over again. If you use only this behaviour, the agent will, in most of the cases, go back and forward through the same door. Therefore this is an example of a behaviour that can only be useful in combination with

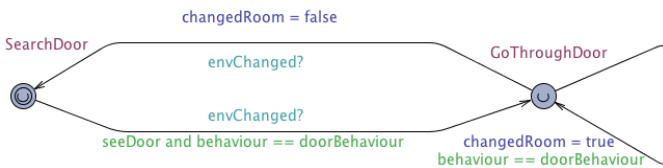


Figure 16. Model of door behaviour

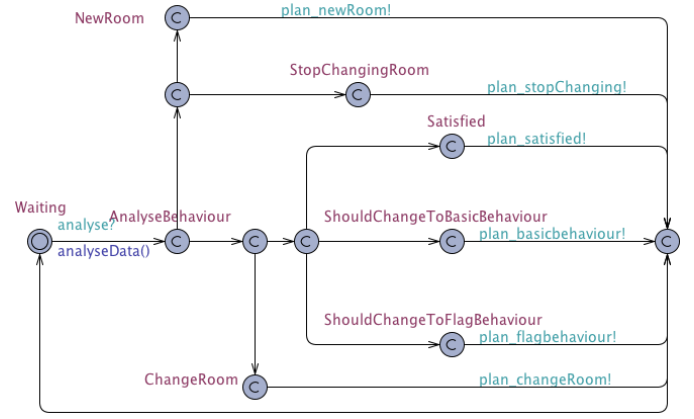


Figure 17. Model of the analyse component in the metabehaviour for a multiple room world

other behaviours. The model for the metabehaviour becomes a bit more complicated. Figure 15 shows that by adding the door behaviour we also add five new edges. When an agent is using basic or flag behaviour and it is searching for packets or a destination for some time, it can change to door behaviour. With door behaviour, an agent tries to change rooms. But if the agent sees its objective (packet or destination) before changing room, it can stop changing. After changing rooms, an agent should go back to basic behaviour or flag behaviour, depending on if the agent has a packet in sight.

Apart from the model for the metabehaviour, we only need to edit the MAPE loop when adding the door behaviour. We will only look into the new analyse component since the other components are more or less self-evident. Figure 17 shows that the analyse-component has become more complex. You can still see the analyse nodes for satisfied, unsatisfied and oversatisfied. Three new possible analysis were added: change room, stop changing, new room. The analyser first checks if the agent uses door behaviour. If it does and the agent entered a new room, the planner is triggered with a matching signal. If the agent did not entered a new room, the analyser will trigger the planner and it will check if the agent should stop changing rooms. Finally, if the agent was not using a door behaviour, the analyser checks if the agent should change rooms. If so, the planner is triggered with the right signal, otherwise we go back to the analysis of the previous analyser.

VI. EVALUATION

We have designed a model of a MAS for the single room packet-world. We extended the model for the multiple room packet-world, by adding a behaviour. We see that the modular approach for situated SA-MAS is not perfectly modular. It is however possible to add a behaviour and several edges. This leads to some consequent changes in the MAPE loop but these changes follow a certain path for each new edge. For example the addition of the *changeRoom* edge, only needs one extra path in the analyse, plan and execute components. In total we added five edges to the metabehaviour model, so we should have added five paths. Since some edges are almost equal, they share a path. We conclude that adding a behaviour only adds extra changes, depending on the new edges. A new edge leads to a possible new path in the MAPE loop.

	This approach		RBM		MBR	
Type	add/rem.	rating	add/rem.	rating	add/rem.	rating
Add	10/0	+	19/0	++	25/0	++
Remove-B	0/3	o	0/5	o	0/9	+
Remove-B+E	0/8	+	0/17	++	0/17	++
Change-Repl	0/0	o	3/3	+	0/0	o
Change-R/A	7/8	++	15/17	++	14/17	++

Table I. TABLE WITH COMPARISON OF EXTRA CHANGES. O IS ALMOST NONE, + MINOR CHANGES, ++ MANY CHANGES.

As a final part of the evaluation, we compare the modularity in different approaches. Role-based modelling (RBM) for multi-agent systems [16] is similar to the metabehaviour. This approach uses roles instead of behaviours. The model-based reconfiguration (MBR) method [19], uses a schema-based modelling approach.

We go over five different types of changes in the approaches. The first type is adding behaviour, *Add*. The second type is removing a behaviour without removing the edges, *Remove-B*. The third type is the removal of a behaviour and the edges its connected with, *Remove-B+E*. The fourth type is the replacement of a behaviour, *Change-Repl*. The fifth and last type is the worst case change, which is removing and adding a new behaviour, *Change-R/A*. The amount extra changes needed is indicated in table I, the amount of design components that were added and removed are indicated accompanied by a rating. This subjective rating has three possibilities: o means almost no changes, + means minor changes, ++ many changes.

The modular approach proposed in this paper, scored generally better in the ratings. This is because the approach only has to add/remove edges in the metabehaviour and depending on these edges, change paths in the MAPE loop. The RBM approach also has role-context diagrams and indirect interaction diagrams. Because of these diagrams, the approach is less modular. The MBR approach needs to do extra changes because of the schema-based models and the different way of self-adaptation.

VII. CONCLUSION

In this paper, we presented a modular approach for situated MAS with self-adaptation. The approach consists of a metabehaviour, behaviours and a MAPE loop which determines when the metabehaviour changes the behaviour it uses. The approach was aimed to be modular. This means that components can be added, removed and changed without a lot of consequent changes in other components.

The approach was evaluated with a case example in the packet-world and with a comparison between other approaches. The approach is not perfectly modular, but in the case example it showed promise. We were able to add a behaviour without creating many additional changes. In the comparison, the modular approach scored relatively well compared to other approaches. The loose coupling between the managed system, metabehaviour and the MAPE loop, is an important reason. A test example and a subjective comparison is not the best evaluation. A better evaluation would have been testing the approach by letting students or other independent persons try the approach. This would provide a better picture of the approach. It would be tested on a wide spectrum of cases. Unfortunately this wasn't possible.

The approach divides behaviour in smaller behaviours and a metabehaviour. Since metabehaviours are behaviours too, the smaller behaviours can be divided in other metabehaviours, creating a tree of metabehaviours. Eventually such tree could represent a really complex behaviour, while still divided in simple and easy to understand components. This is however something for future research.

REFERENCES

- [1] J. Bengtsson and W. Yi. "Timed Automata: Semantics, Algorithms and Tools". In Lectures on Concurrency and Petri Nets, pages 87-124. Springer, 2004.
- [2] D. G. de la Iglesia and D. Weyns. "MAPE-K formal templates to rigorously design behaviors for self-adaptive systems". ACM Trans. Auton. Adapt. Syst. 10, 3, 2015.
- [3] D. G. de la Iglesia and D. Weyns. "SA-MAS: Self-adaptation to enhance software qualities in multi-agent systems". In Proceedings of the 2013 international conference on Autonomous agents and multi-agent systems. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1159-1160, 2013.
- [4] J. Kephart and D. Chess, "The vision of autonomic computing", IEEE Comp. Soc., vol. 36, no. 1, 2003.
- [5] M. U. Iftikhar and D. Weyns. "ActivFORMS: Active formal models for self-adaptation." In Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. ACM, New York, NY, USA, 125-134, 2014.
- [6] D. Sansores, J. Pavon, "An Adaptive Agent Model for Self-Organizing MAS", Proceedings of 7th Int. Conf. on Autonomous Agents and Multiagent Systems, pp. 1639-1642, 2008.
- [7] P. Vromant, D. Weyns, S. Malek, and J. Andersson, "On Interacting Control Loops in Self-Adaptive Systems". In Proceedings of the 6th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. ACM, New York, NY, USA, 202-207, 2011.
- [8] M. U. Iftikhar and D. Weyns, "A Case Study on Formal Verification of Self-Adaptive Behaviors in a Decentralized System", EPTCS, vol.91, 2012.
- [9] D. Weyns and M. Georgeff, "Self-Adaptation using Multiagent Systems", in IEEE Software, vol. 27, no. 1, pp. 86-91, Jan.-Feb. 2010.
- [10] D. Weyns, S. Malek et al., "FORMS: a formal reference model for self-adaptation". In Proceedings of the 7th international conference on Autonomic computing. ACM, New York, NY, USA, 205-214, 2010.
- [11] D. Weyns, A. Helleboogh and T. Holvoet, "The Packet-World: A Test Bed for Investigating Situated Multi-Agent Systems", chapter from a book: Software Agent-Based Applications, Platforms and Development Kits, pp. 383-408, available: <http://www.springerlink.com>, 2005.
- [12] S. Lynch, "Using Meta-Agents to Build MAS Platforms and Middleware". In Proceedings of the 3rd International Conference on Agents and Artificial Intelligence, pages 385-388, 2011.
- [13] D. Weyns, M. U. Iftikhar, and Joakim S. oderlund. "Do external feedback loops improve the design of self-adaptive systems? A controlled experiment". In Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, 2013.
- [14] C. IBM, "An architectural blueprint for autonomic computing". Tech. Rep. IBM, 2003.
- [15] G. Behrmann, A. David, P. Pettersson, W. Yi, and M. Hendriks, "UPPAAL 4.0", In Quantitative Evaluation of Systems, 2006.
- [16] E. Steegmans, "Role-Based Modeling for Multi-Agent Systems". KULeuven PHD thesis, 2011.
- [17] W. Jamroga, A. Męski, M. Szreter: "Modularity and openness in modeling multi-agent systems". In Puppis, G., Villa, T., eds.: Fourth International Symposium on Games, Automata, Logics and Formal Verification. Volume 119., EPTCS, 2013.
- [18] R. Alur, C. Courcoubetis, and D.L. Dill. "Model-checking for real-time systems". In Proceedings, Seventh Annual IEEE Symposium on Logic in Computer Science, pages 414-425. IEEE Computer Society Press, 1990.
- [19] A. Goel, E. Stroulia, Z. Chen, and P. Rowland. "Model-based reconfiguration of schema-based reactive control architectures". In Proceedings of the AAAI Fall Symposium on Model-Based Autonomy, 1997.